

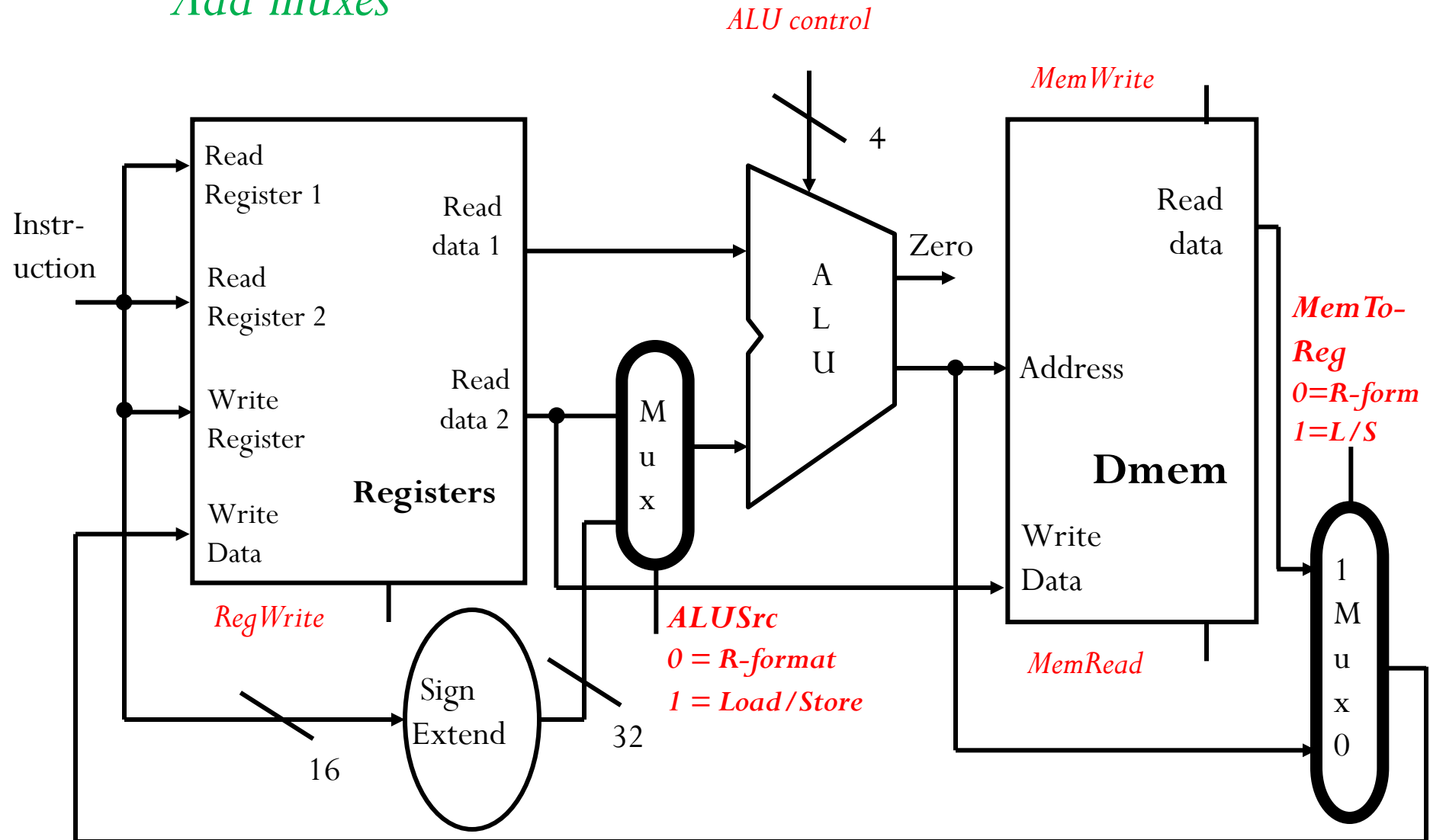
Computer System Architecture

Datapath & Control
(Microarchitecture)

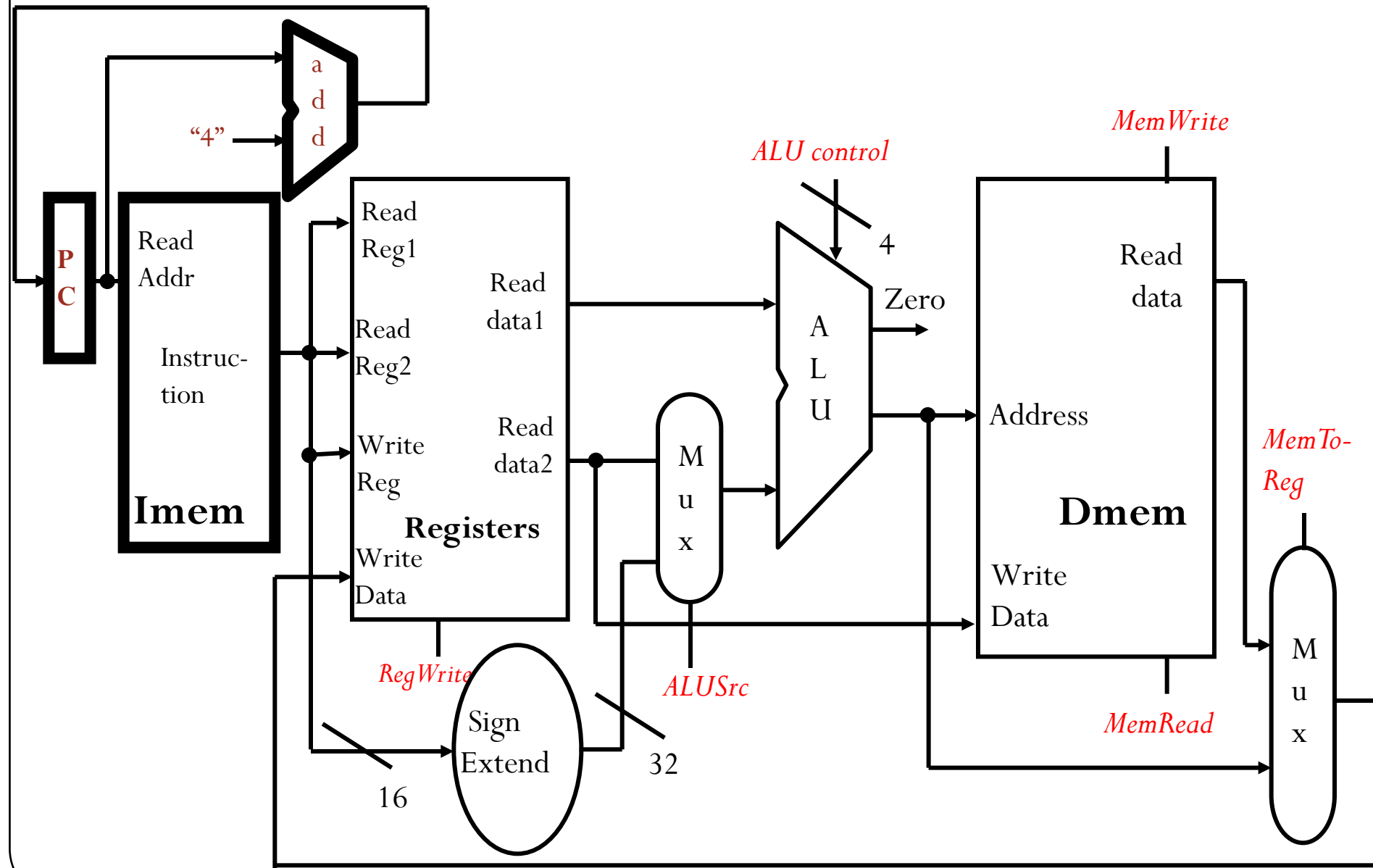
Single Cycle vs. Multi Cycle

Datapath Step 3-5: R-form + Load/Store

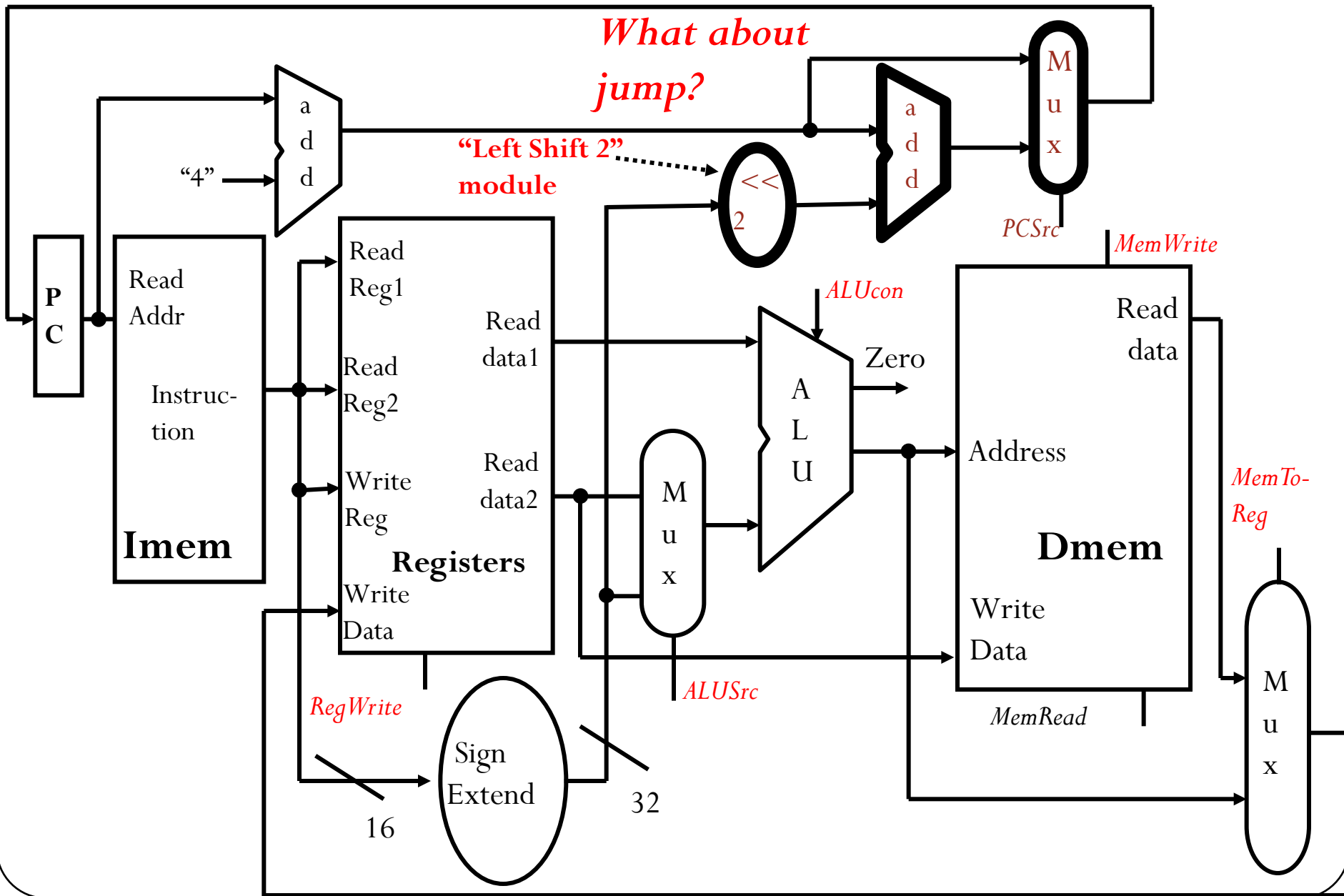
Add muxes



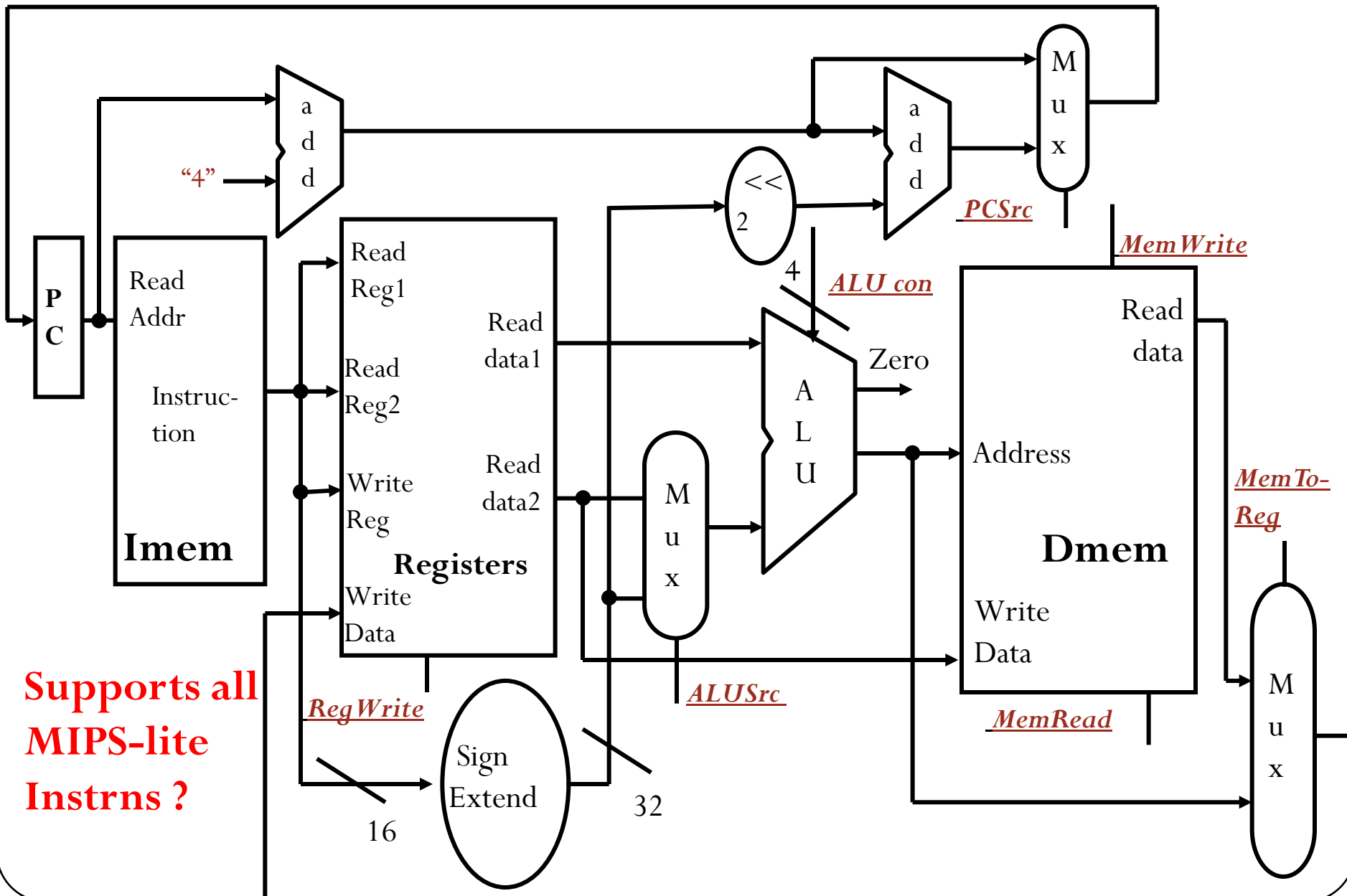
Compose Datapath: ... + Imem + PC



Compose Datapath: ... + Branch



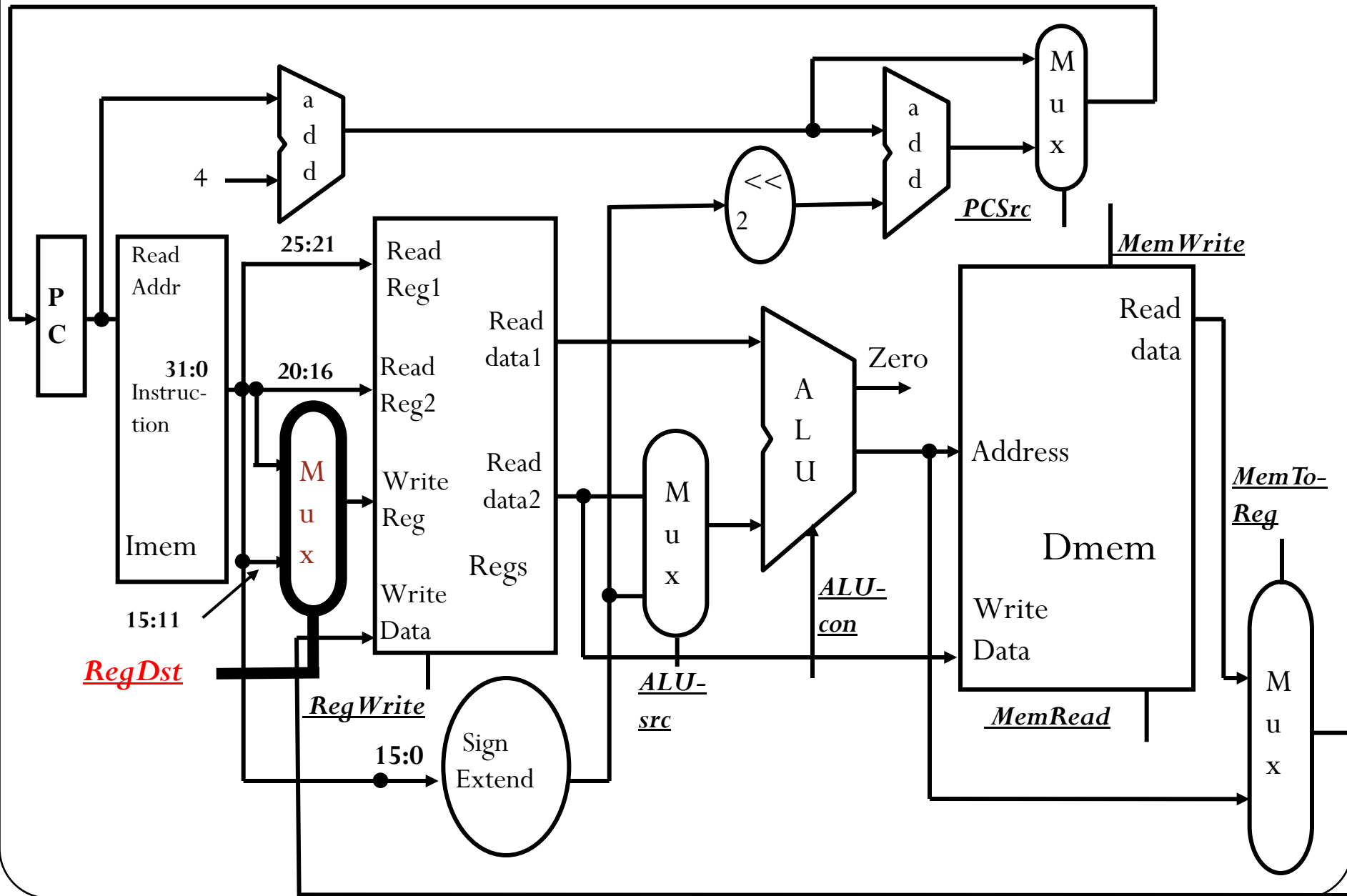
Recap: Single-Cycle Datapath



Datapath: Additional Operands

- Wrinkle: destination registers may differ across instruction formats:
 - R-format: $[rd] \leftarrow [rs] \text{ op } [rt]$
add \$t0,\$s0,\$s1, For this instn, bits 11-15 is destination (t0), which should be connected to the **write reg**.
 - I-format: $[rt] \leftarrow \text{mem}[[rs] + \text{imm16}]$
lw \$t0,24(\$s3), For this instn, bits 16-20 should go to the **write port**. Bits 0-15 go to the ALU as address.
 - **Connection to the write port changes!**
- **Solution? Mux to the rescue!**

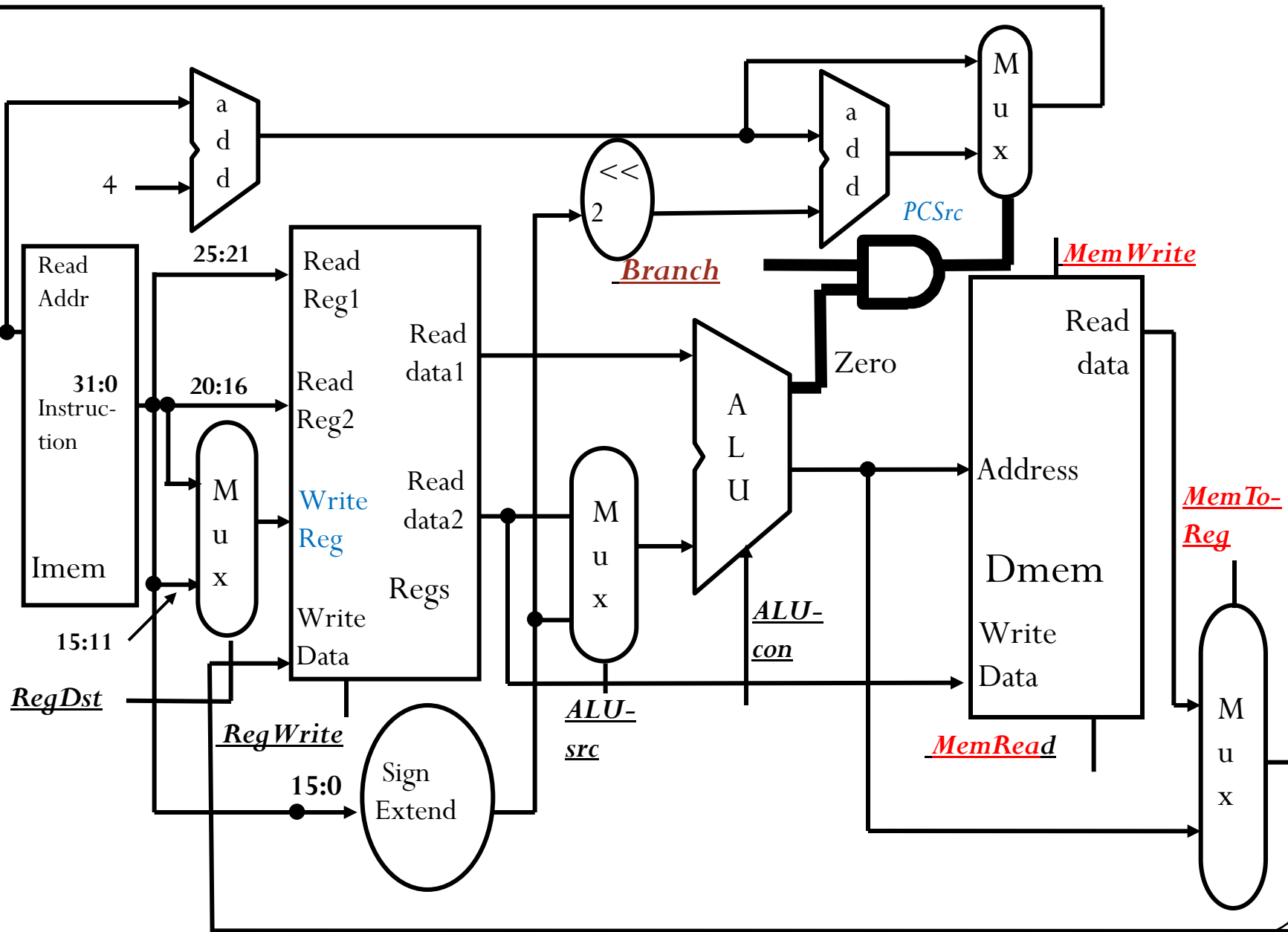
Datapath :add RegDst Mux



Datapath: Conditional Branch

- What if instruction is a conditional branch (**beq**)?
 - if operands equal, take branch (PC gets $PC+4*\text{offset}$)
 - else PC gets $PC+4$
- Therefore, set control point PCSrc = 1 if and only if **beq** and **Zero asserted**

Datapath : add Branch control point



Processor = Datapath + Control

- CPU = Datapath + Control
- Single-Cycle Design:
 - Instruction takes exactly one clock cycle
 - Datapath units used only once per cycle
 - Writable state updated at end of cycle
- What must be “controlled”?
 - Multiplexors (Muxes)
 - Writable state elements: Register File, Data Memory (Dmem)
 - *what about PC? Imem?*
 - ALU (which operation?)

Processor = Datapath + Control

- **Single-Cycle Design**: everything happens in one clock cycle

⇒

until next falling edge of clock, processor just one big
combinational circuit!!!

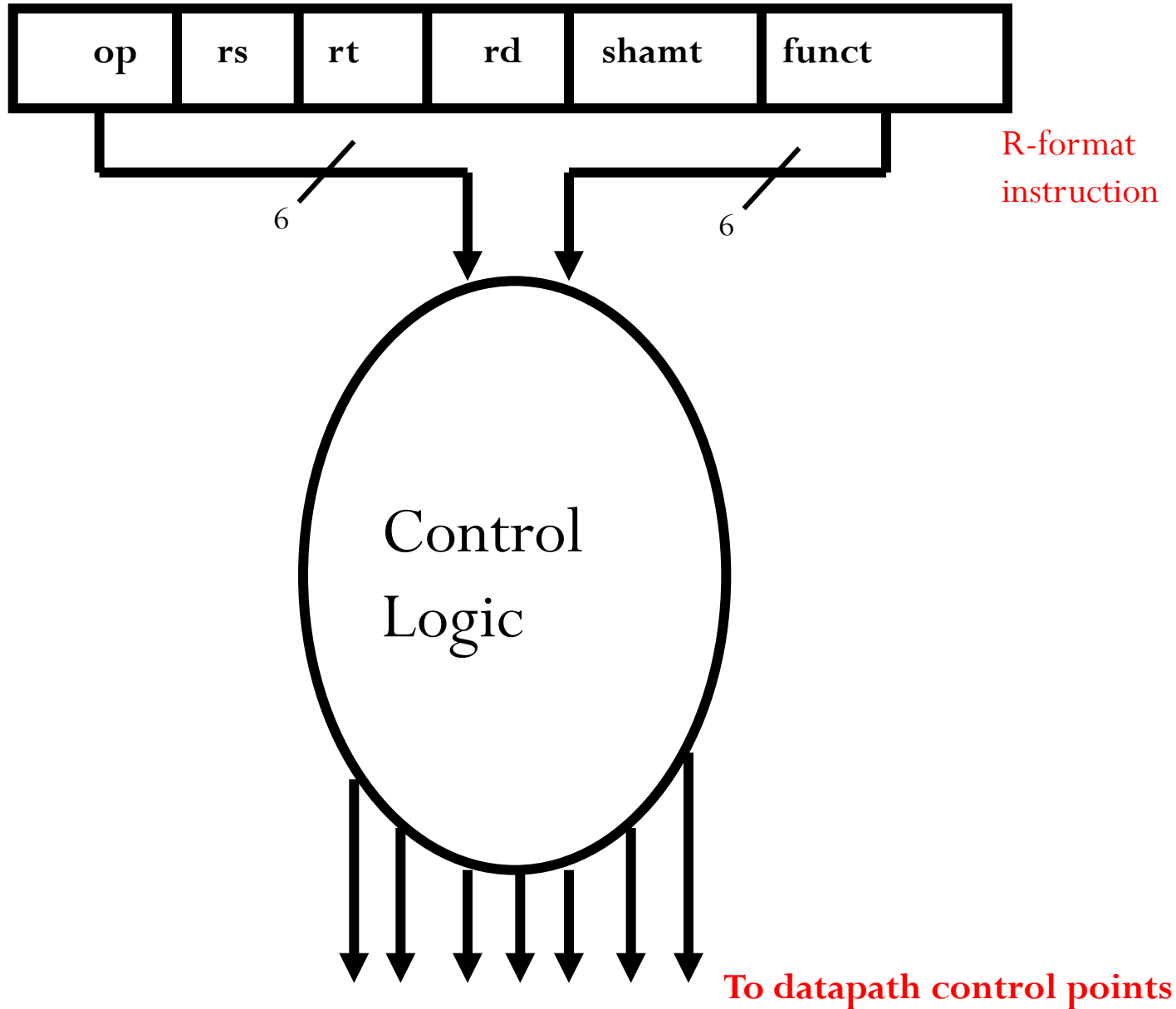
⇒

control is just a combinational circuit

(output, just function of inputs)

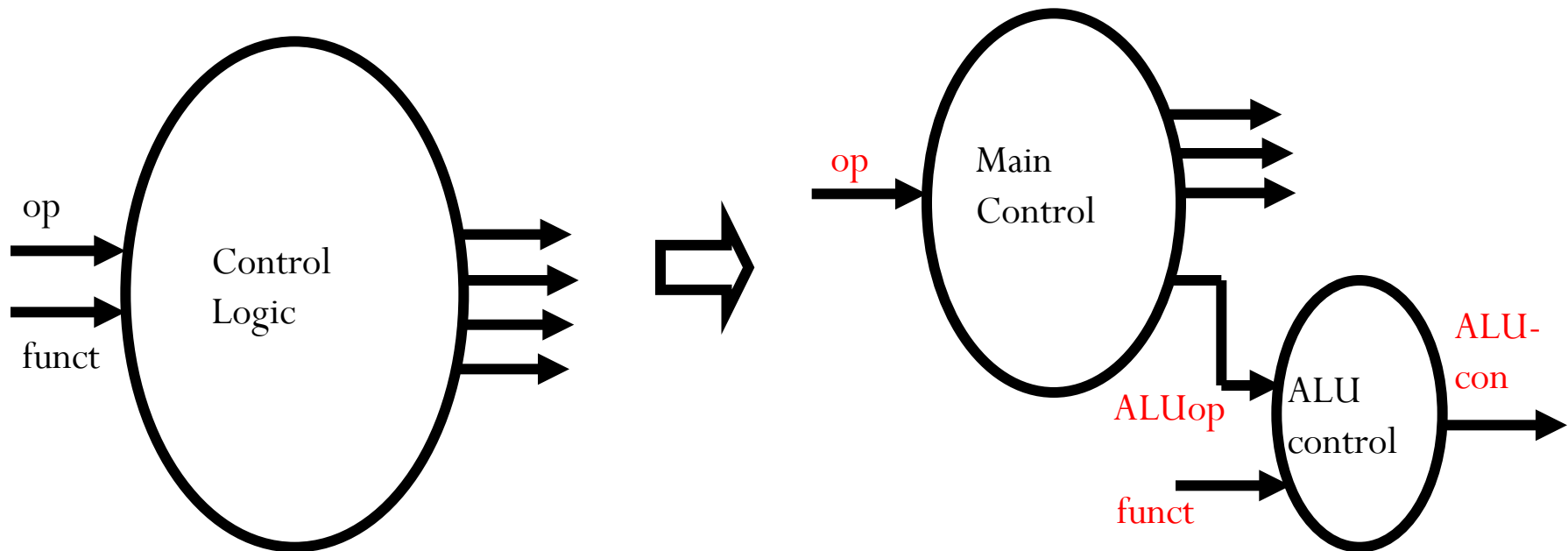
- **outputs?** control points in datapath
- **inputs?** the current instruction! (opcode, funct control everything)

Defining Control

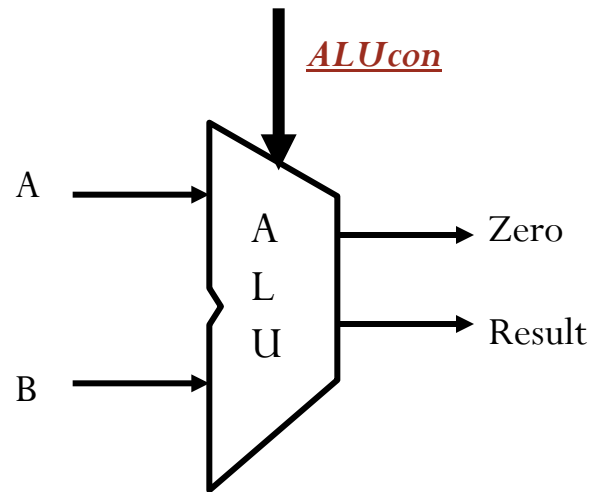


Defining Control

- Note that **funct field only present in R-format instruction**
 - **funct controls ALU only**
- To simplify control, define Main and ALU control separately – using multiple levels will also increase speed – important optimization technique
- **ALUop intermediate inputs** will be defined



Defining ALU Control



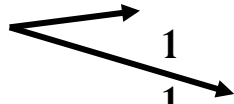
ALUcon	ALU function	Instruction(s) supported
0000	AND	R-format (and)
0001	OR	R-format (or)
0010	add	R-format (add), lw, sw
0110	subtract	R-format (sub), beq
0111	set on less than	R-format (slt)
1100	NOR	R-format (nor)

Defining ALU Control

Instruction opcode	Desired ALU Action	ALUOp	funct	ALUcon
lw	add	00	xxxxxx	0010
sw	add	00	xxxxxx	0010
beq	subtract	01	xxxxxx	0110
R-type	add	10	100000 (add)	0010
R-type	subtract	10	100010 (sub)	0110
R-type	logical AND	10	100100 (and)	0000
R-type	logical OR	10	100101 (or)	0001
R-type	set on less	10	101010 (slt)	0111

ALUOp		Funct Field							ALUcon
a1	a2	f5	f4	f3	f2	f1	f0		
Don't Cares	0	0	x	x	x	x	x	x	0010
	x	1	x	x	x	x	x	x	0110
	1	x	x	x	0	0	0	0	0010
	1	x	x	x	0	0	1	0	0110
	1	x	x	x	0	1	0	0	0000
	1	x	x	x	0	1	0	1	0001
	1	x	x	x	1	0	1	0	0111

Don't
Cares



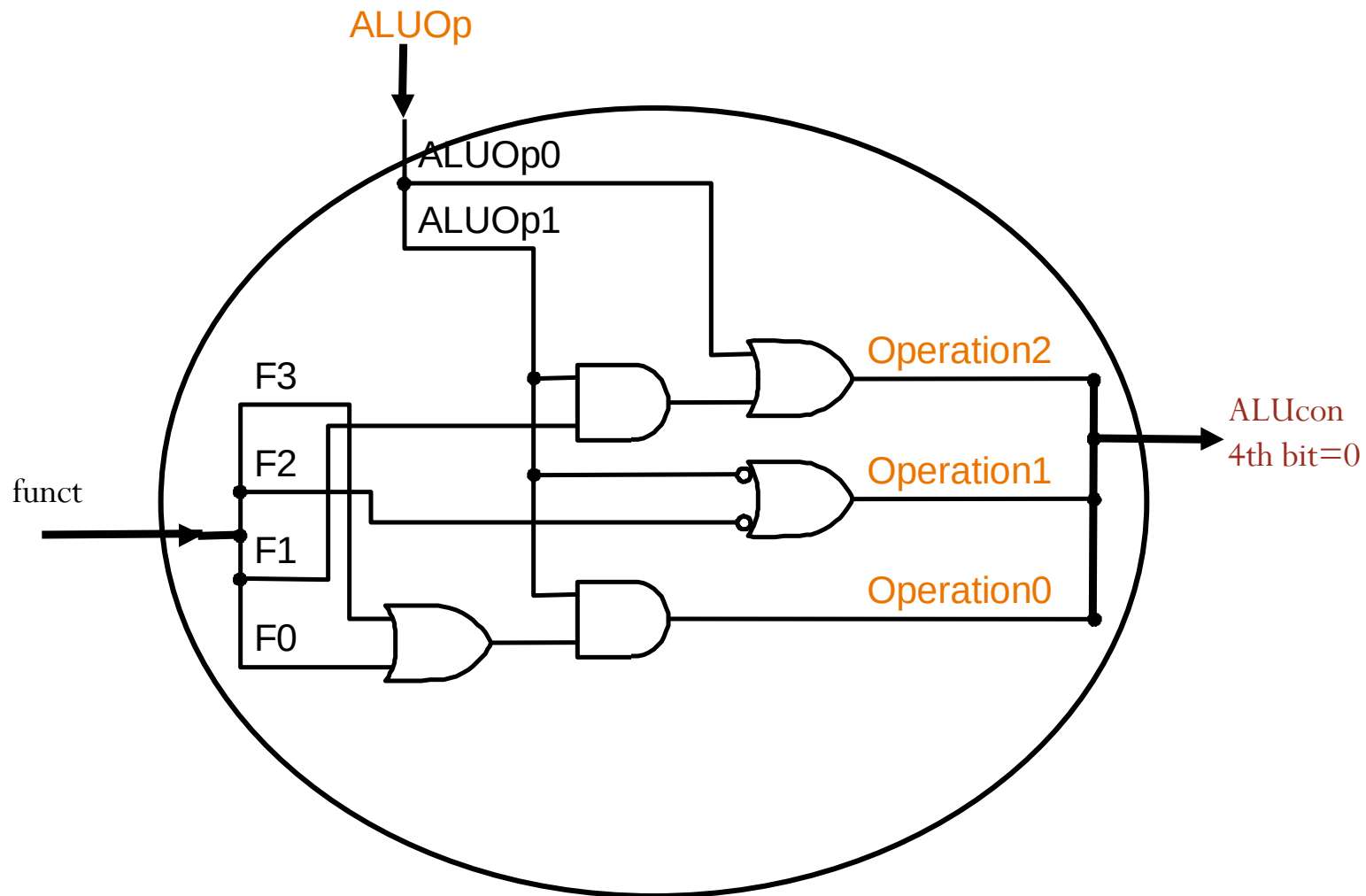
Deriving ALUcon outputs

From the truth table, output signals can be easily derived because of don't cares.

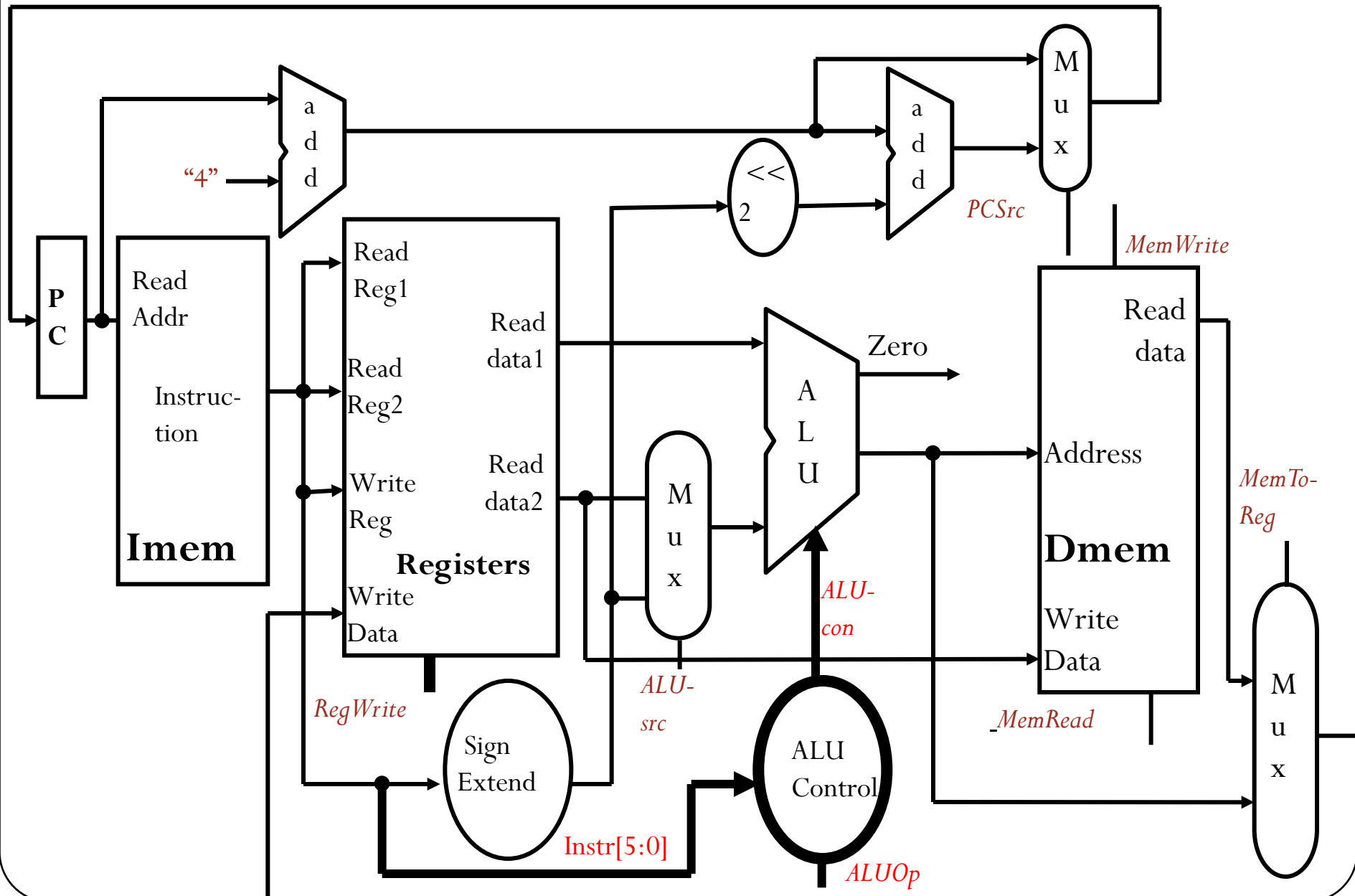
- Operation2 (**msb**) = $\text{ALUOp0 OR (ALUOp1 AND F1)}$
- Operation1 = ALUOp1 NOR F2
- Operation0 (**lsb**) = $\text{ALUOp1 AND (F3 OR F0)}$

ALUOp is supplied by the main control unit (to be designed) and F0-F5 are the lsbs of the instruction.

Fully Minimized ALU Control



Datapath + ALU Control



Implementing Main Control

Main Control has one 6-bit input, 9 outputs (7 are 1-bit, ALUOp is 2 bits)

To build Main Control as **sum-of-products**:

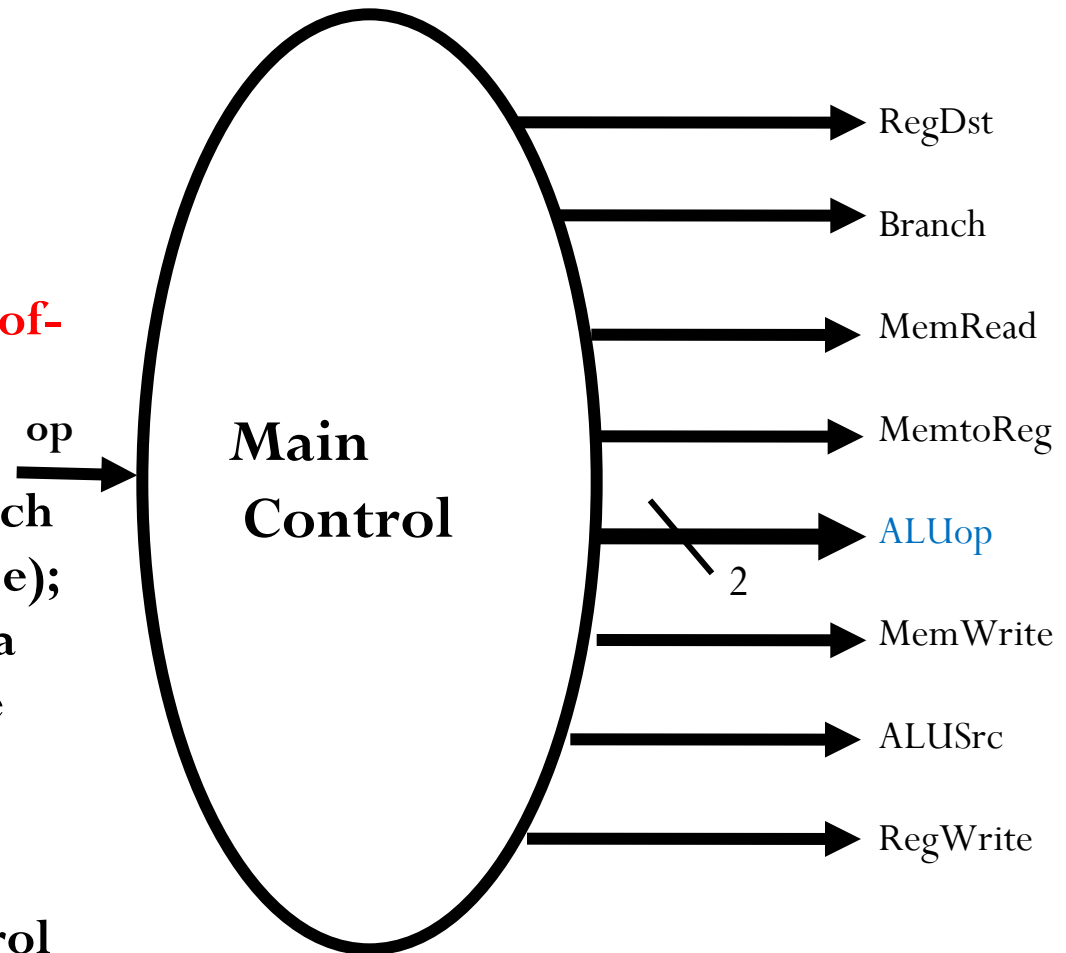
(1) Construct a minterm for each different instruction (or R-type); each minterm corresponds to a single instruction (or all of the R-type instructions),

e.g., $M_{R-format}$, M_{lw}

(2) Determine each main control output by forming the logical OR of relevant minterms

(instructions), e.g., RegWrite:

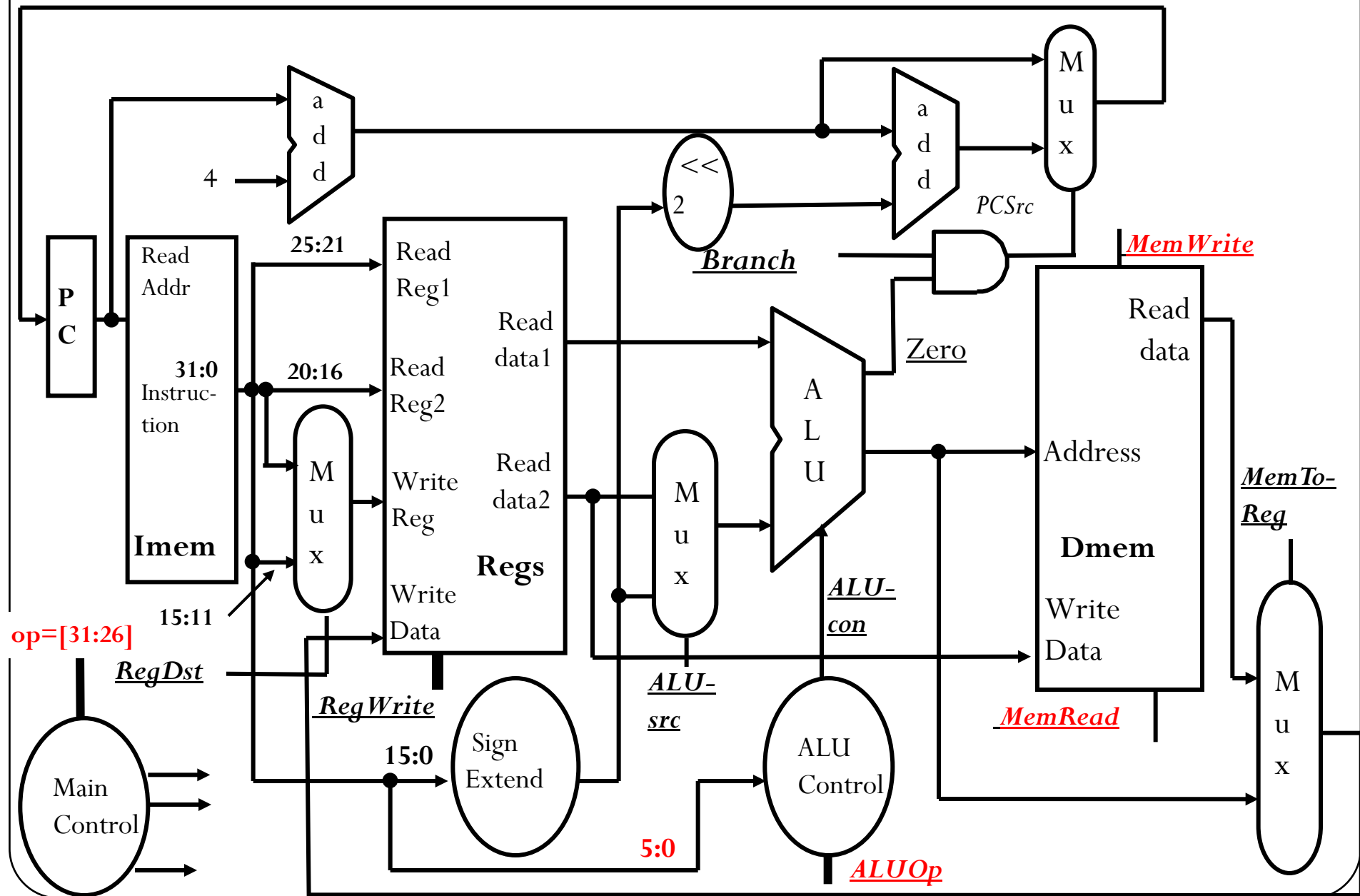
$M_{R-format}$ OR M_{lw}



Setting Control Lines Depend on Opcode

Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

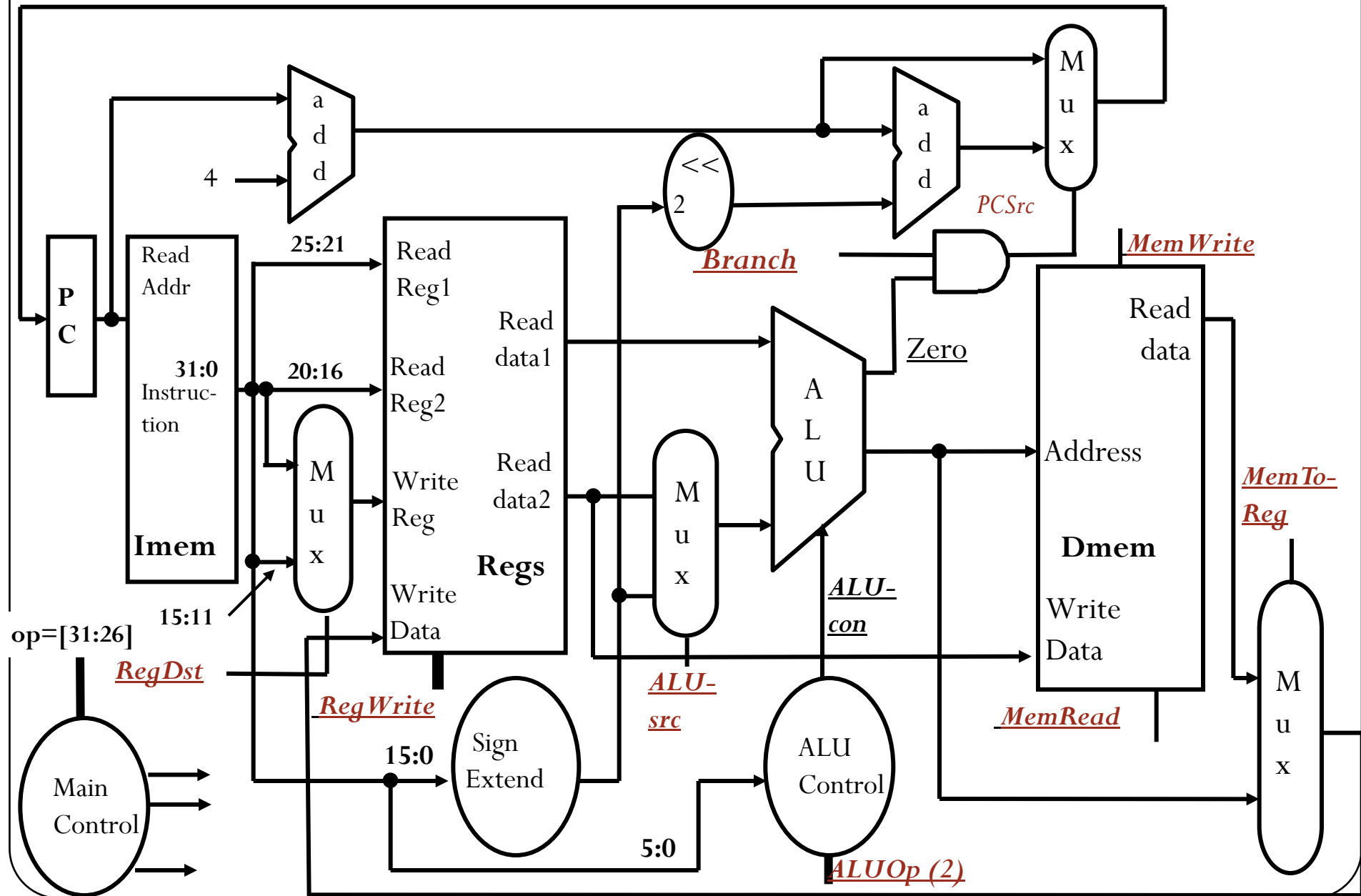
Single-Cycle MIPS-lite CPU



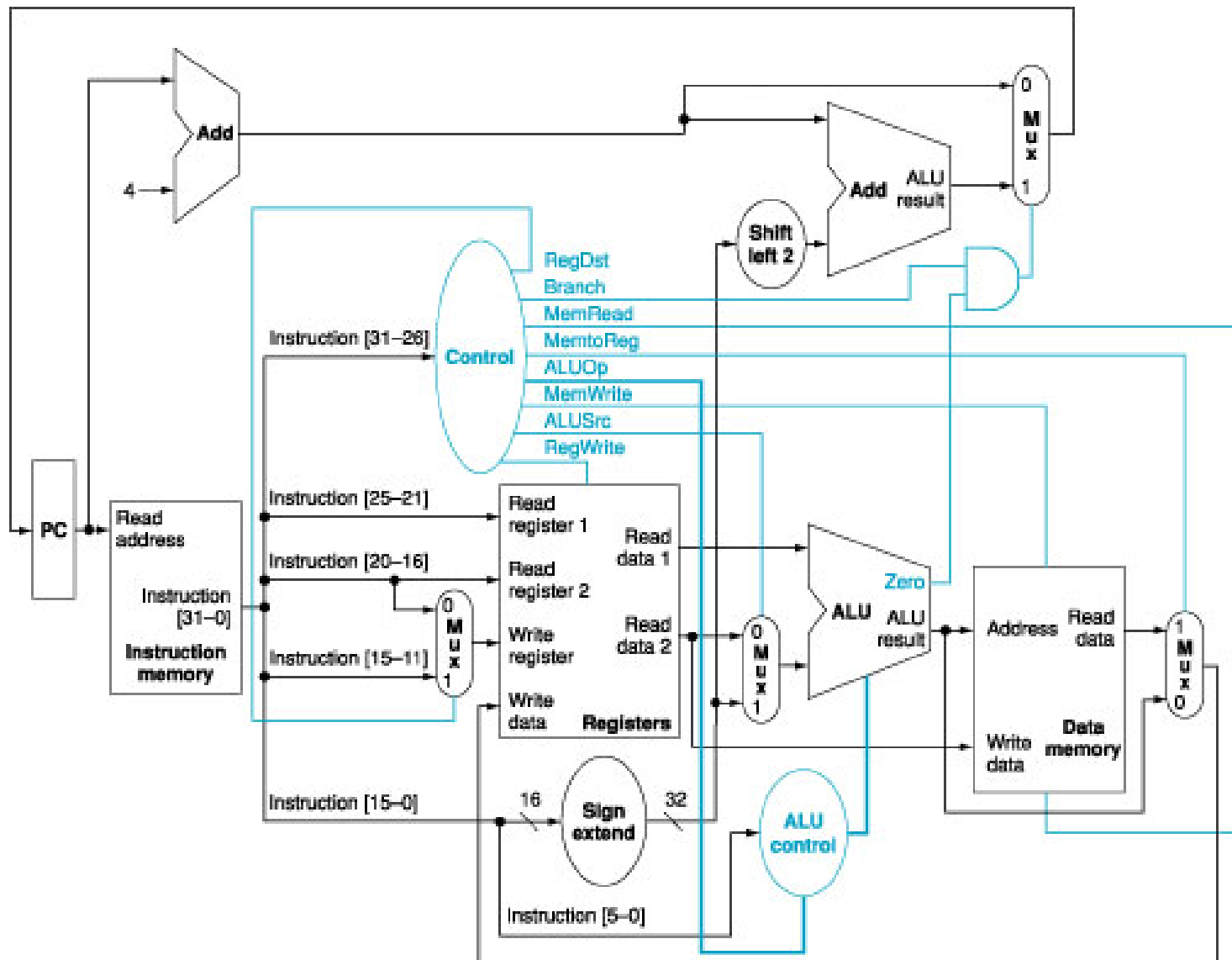
Adding Jal instrn to datapath

1. Jal consists of jump and link for function calls — jump to the function address (j target) already implemented
2. Place $PC+4$ (return address) in $\$ra$. So
 - (a) extend the mux at reg input to include $\$31$ ($\$ra$) as another input and
 - (b) expand the mux at write data input to have $PC+4$ as another input.

Single-Cycle MIPS-lite CPU



Datapath with Control Signals

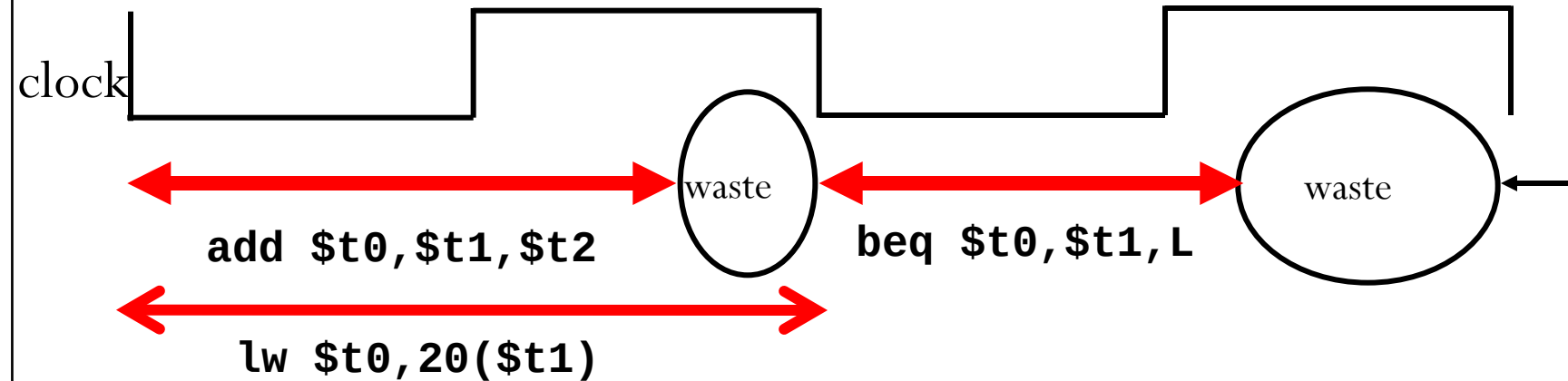


Single-Cycle Machine: Appraisal

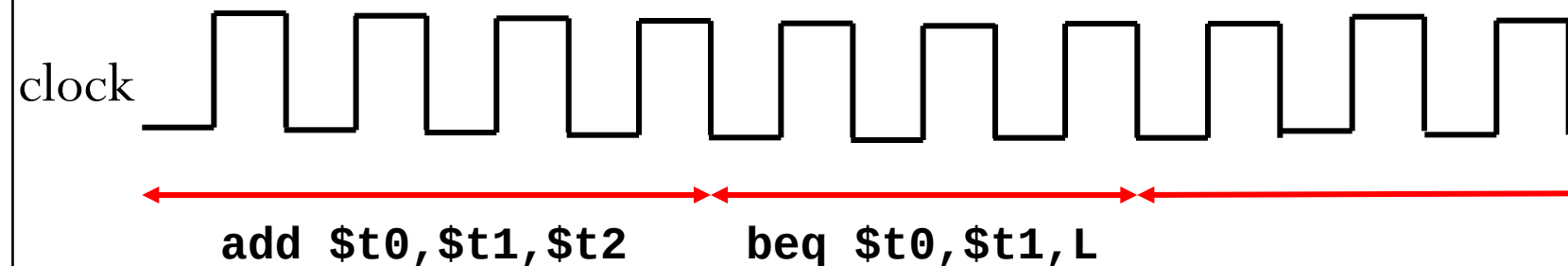
- All instructions complete in **one clock cycle** ($\text{CPI} = 1$)
- Some instructions take **more steps** than others
 - lw is most expensive (5 steps, vs. 4 for R-type and sw, 3 for beq)
- **Clock cycle must cover longest instruction $\Rightarrow$ inefficient**
 - suppose **mul** is added?
 - 32-shift/add steps $\Rightarrow$ would delay every other instruction

Recap: Clocking: single-cycle vs. multicycle

Single-cycle Implementation



Multicycle Implementation



- **Multicycle Implementation:**
less waste=higher performance

Example

- Assume 2ns for instruction, 1ns for decode/register read, 2ns for ALU and 1 ns for register write, 2ns for data memory (5 Steps).
- Single-cycle datapath clock period = 8 ns.
- Assume an instn mix of 24% loads, 12% stores, 44% R-format, 18% branches, and 2% jumps.
- Assuming a variable-cycle datapath, average clock period can be = 6.3 ns.
- Possible Speed-up = 1.7 ns

Multicycle Implementation (MIPS-lite v.2)

- Want more efficient implementation
- Each step will take one clock cycle (not each instruction)
[CPI > 1]

⇒ shorter clock cycle: cycle time **constrained by longest step, not longest instruction**

- simpler instructions take fewer cycles

⇒ higher overall performance

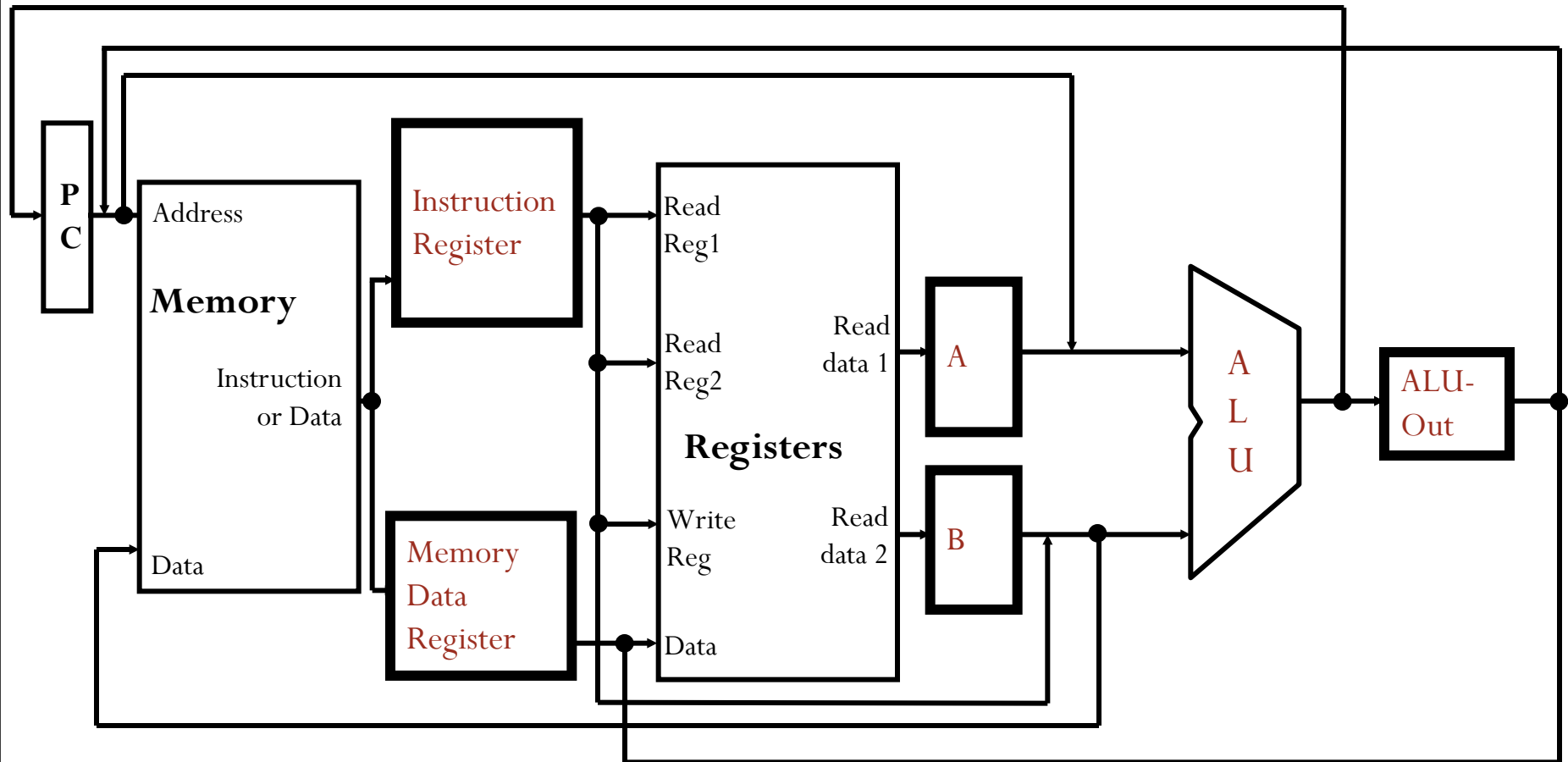
- complex control: finite state machine/
Microprogramming
- Versatile (can extend for new instructions: add3, swap, etc.)

Recap: How fast can we run the clock?

- Depends on how much want done per clock cycle
 - Can do: **several** “inexpensive” datapath operations per clock
 - simple gates (AND, OR, ...)
 - single datapath registers (PC)
 - sign extender, left shifter, multiplexor
 - PLUS: exactly **one** “expensive” datapath operation per clock
 - ALU operation
 - Register File access (2 reads, or 1 write)
 - Memory access (read or write)

Multicycle Datapath (overview)

*MIPS-lite
Multicycle Version*

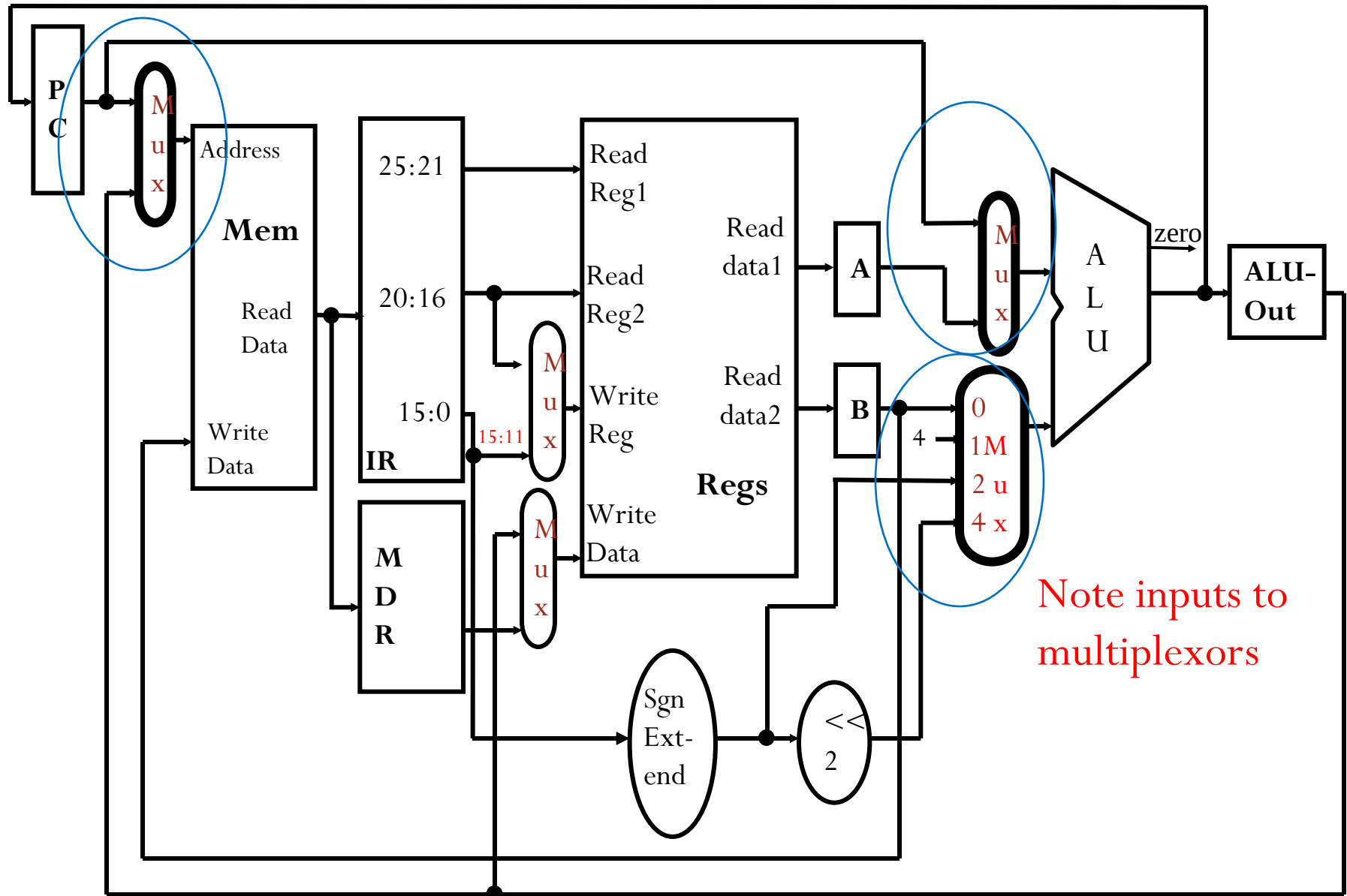


- *One ALU (no extra adders)*
- *One Memory (no separate Imem, Dmem)*
- *New Temporary Registers (“clocked” / require clock input)*

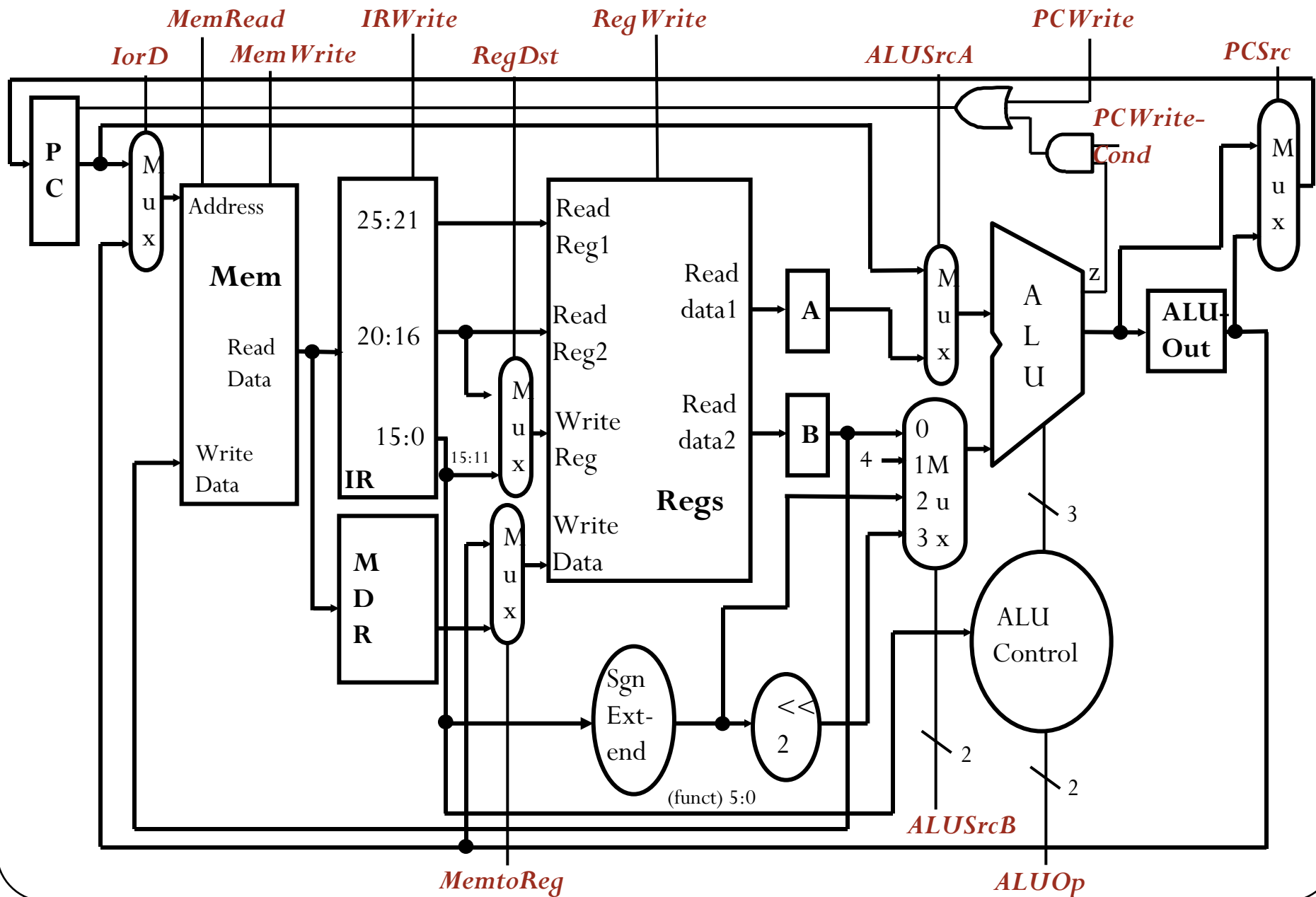
Multicycle Implementation

- Datapath changes
 - one memory: both instructions and data (**because** can access on **separate steps**)
 - one ALU (eliminate extra adders)
 - extra “invisible” registers to capture intermediate (per-step) datapath results
 - Controller changes
 - controller must fire control lines in *correct sequence and correct time*
- ⇒ controller **must remember current execution step**, advance to next step. **Why ?**

Multicycle Datapath: Add Multiplexers



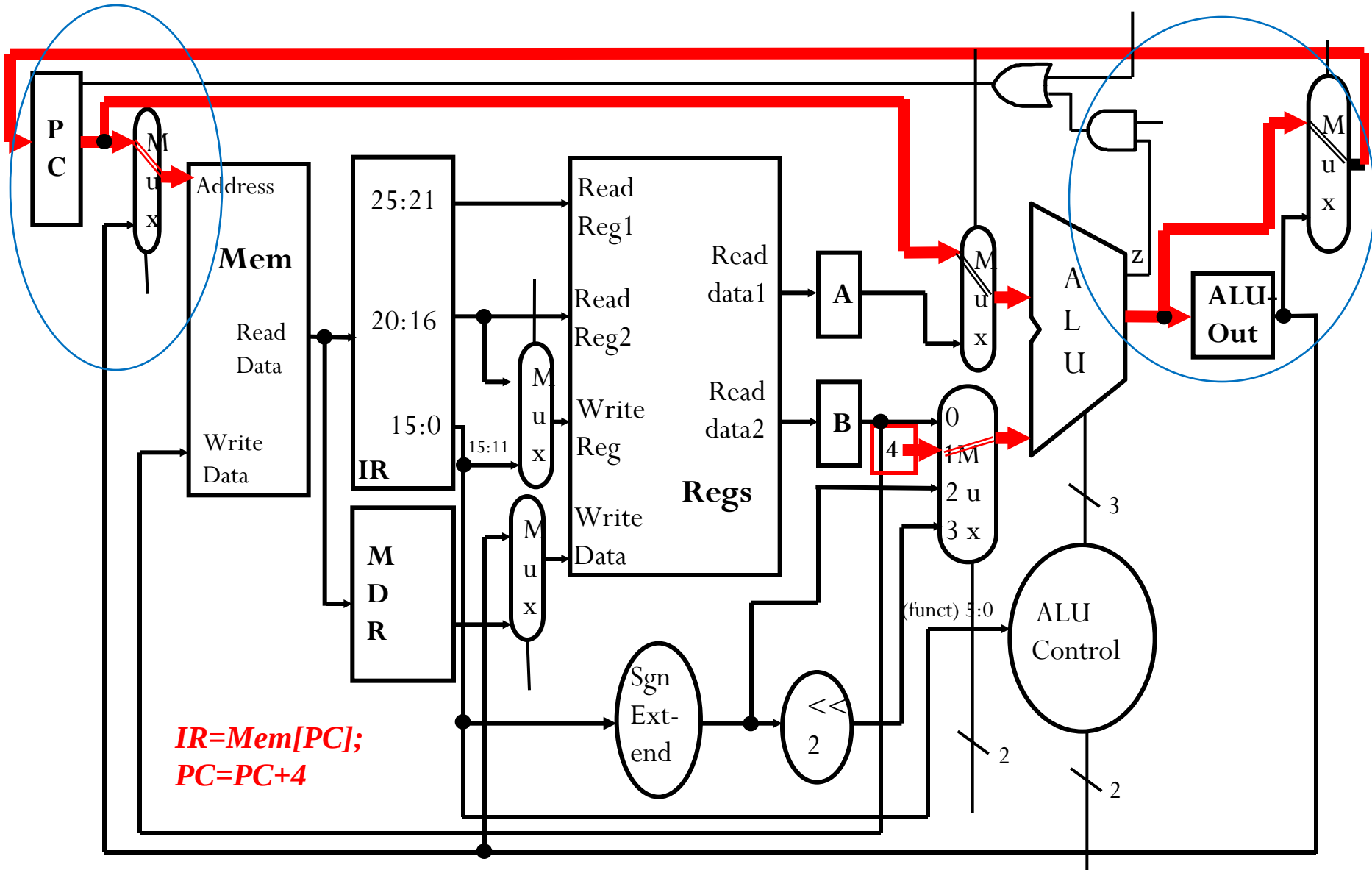
Datapath + Control Points



Multicycle Instruction Cycle

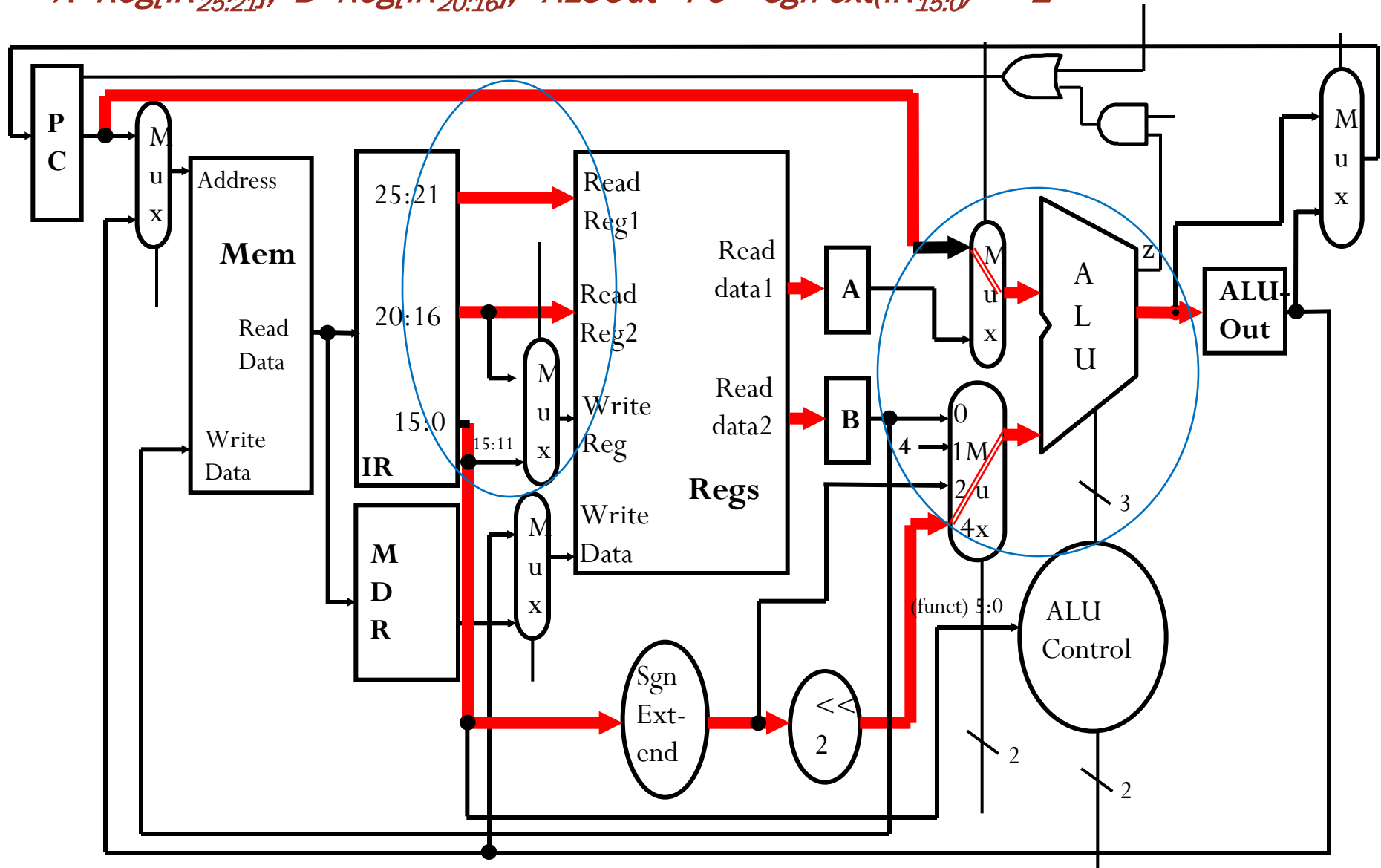
- All instructions execute in 3-5 cycles
 - 3 cycles: beq
 - 4 cycles: R-type, SW
 - 5 cycles: lw
- 1: fetch instruction, $PC = PC + 4$
- 2: decode, fetch registers, compute branch target
- 3: execute / compute access address / branch
- 4: access memory / complete R-type
- 5: (lw) store memory

Cycle 1- Fetch: $IR = Mem[PC]; PC = PC + 4$

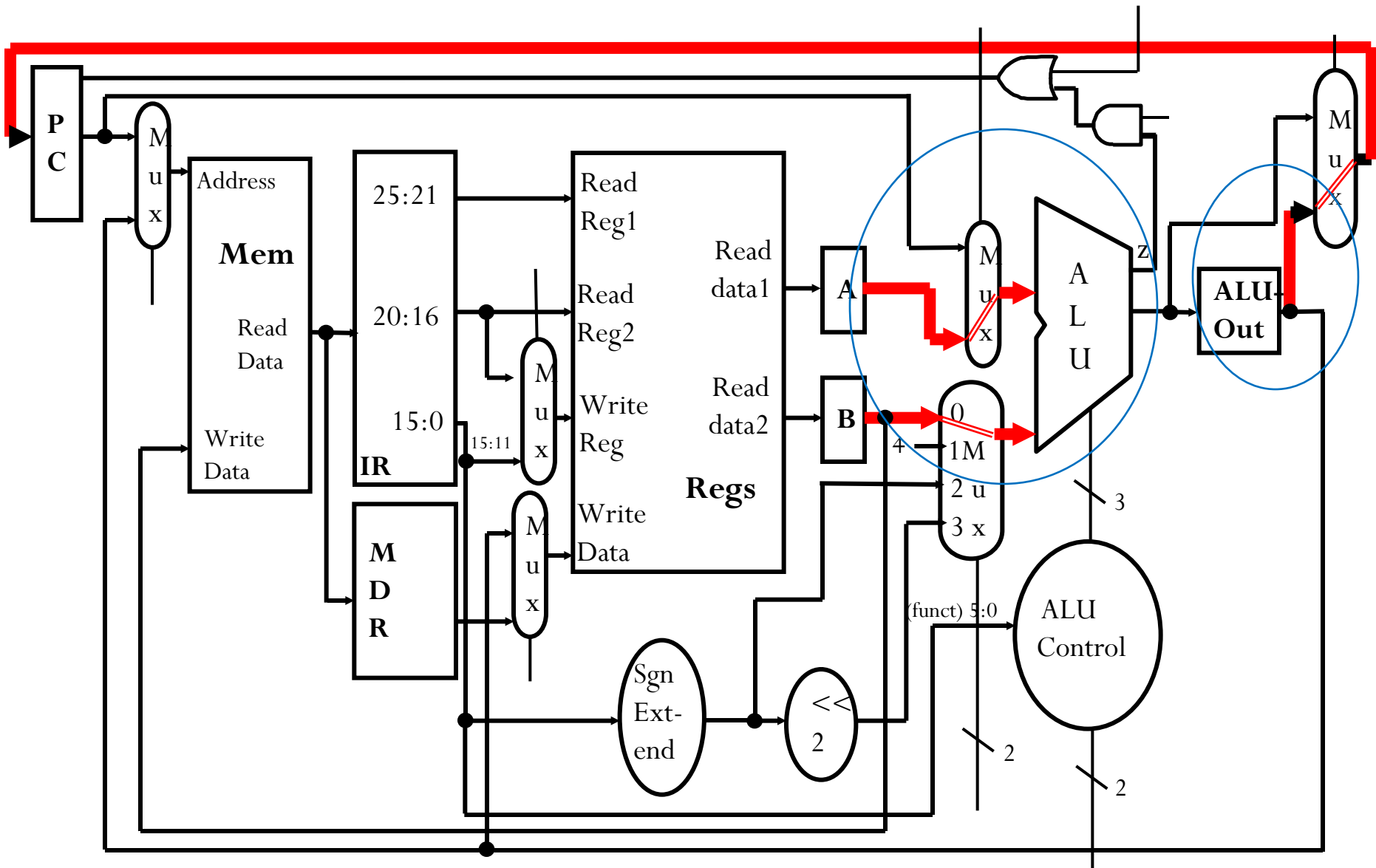


Cycle 2, Decode (Reg or Jump) :

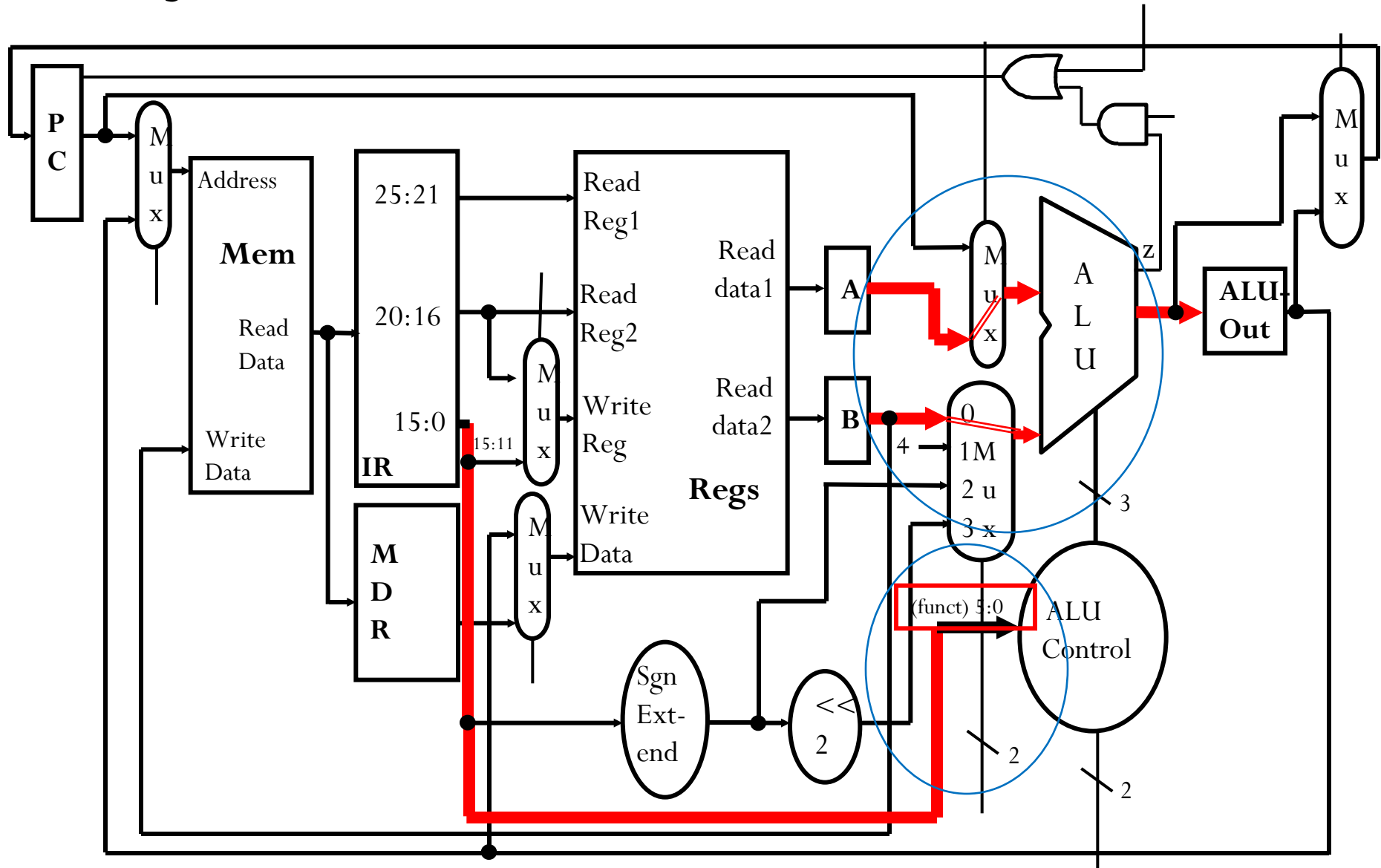
$A = \text{Reg}[IR_{25:21}]$; $B = \text{Reg}[IR_{20:16}]$; $\text{ALUOut} = \text{PC} + \text{sgn-ext}(IR_{15:0}) \ll 2$



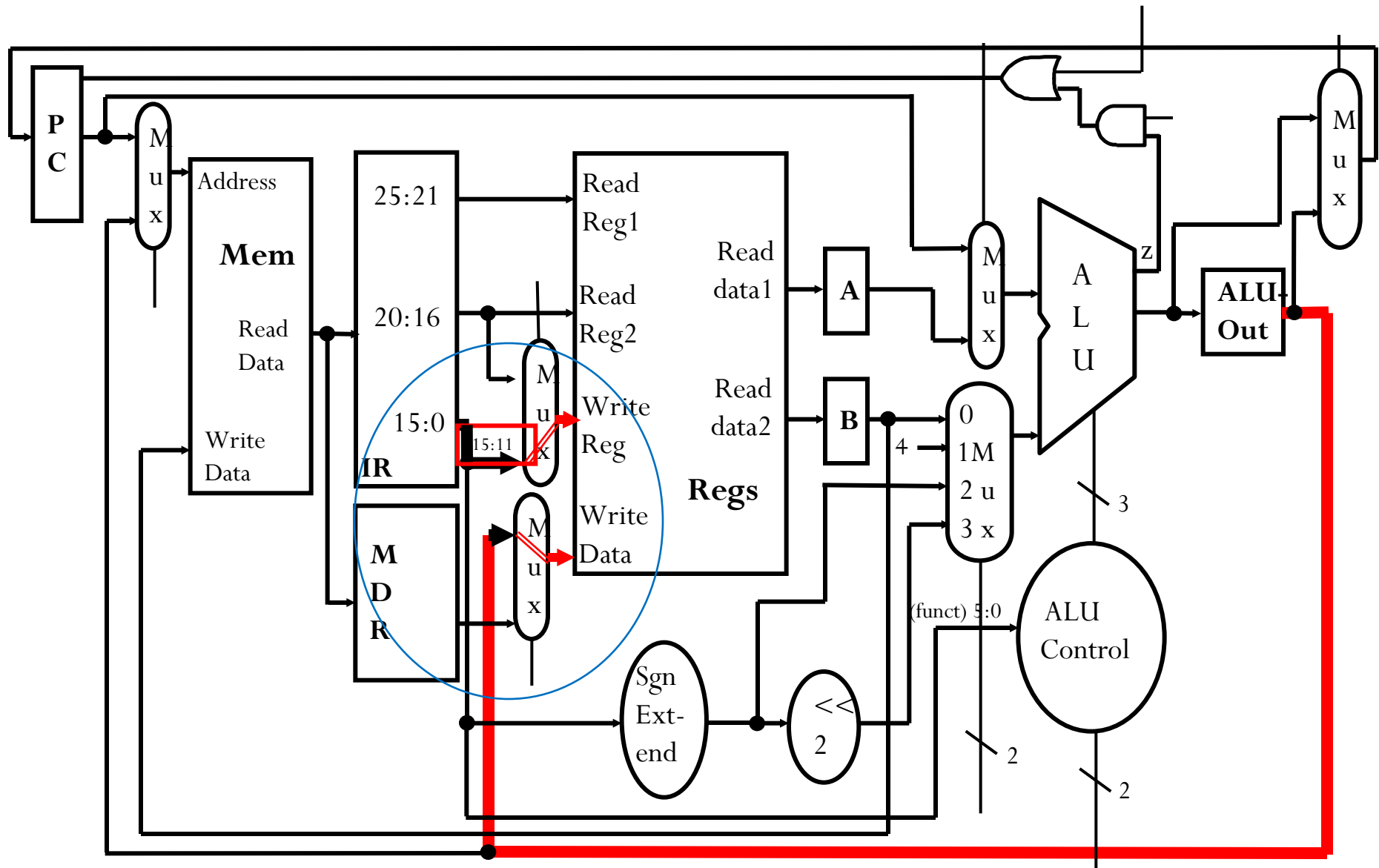
Cycle 3 Exec beq: *if (A==B) PC =ALUOut*



Cycle 3, Exec R-format: $ALUOut = A \text{ op } B$

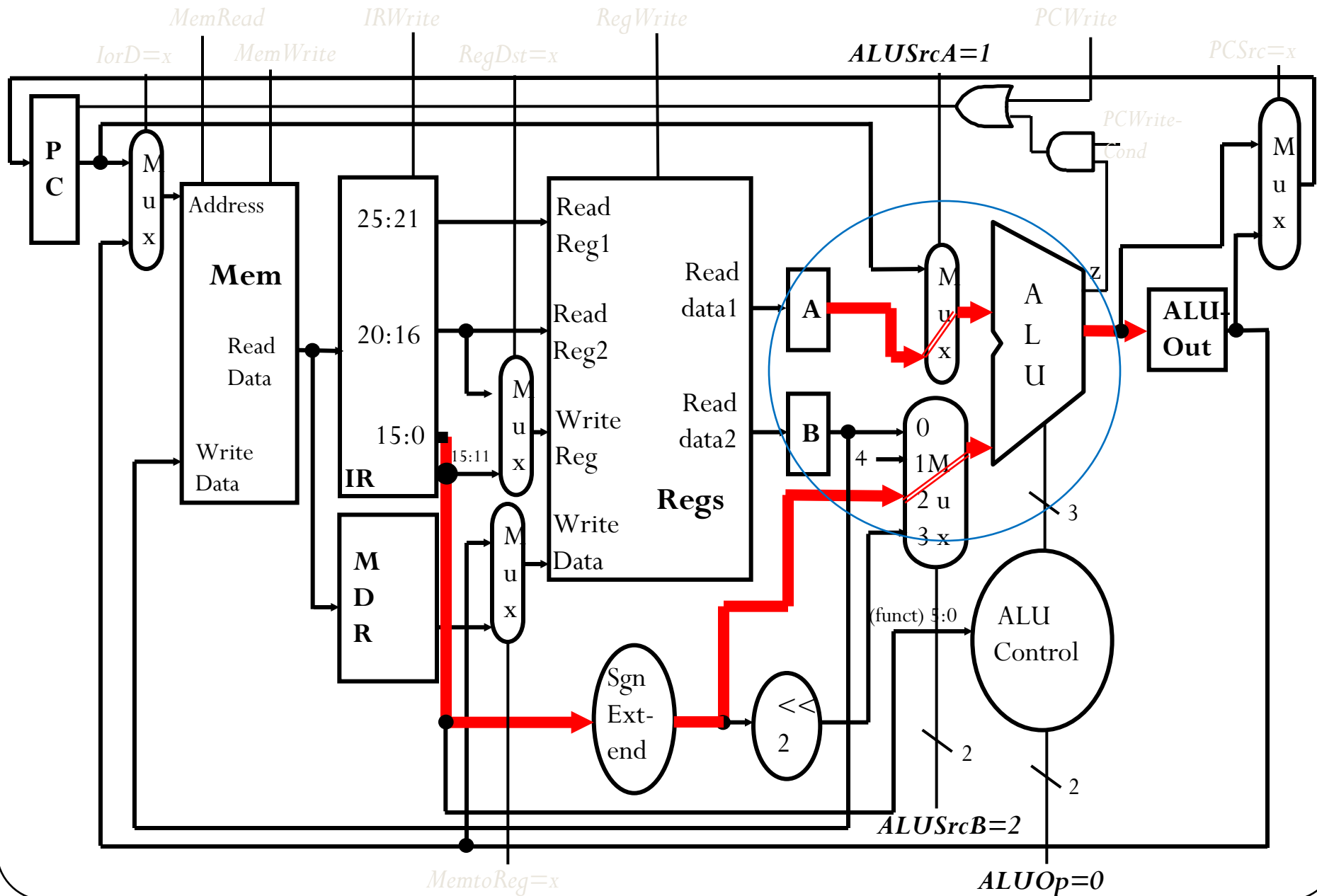


Cycle 4 Store R-format: $Reg[IR_{15:11}] = ALUOut$

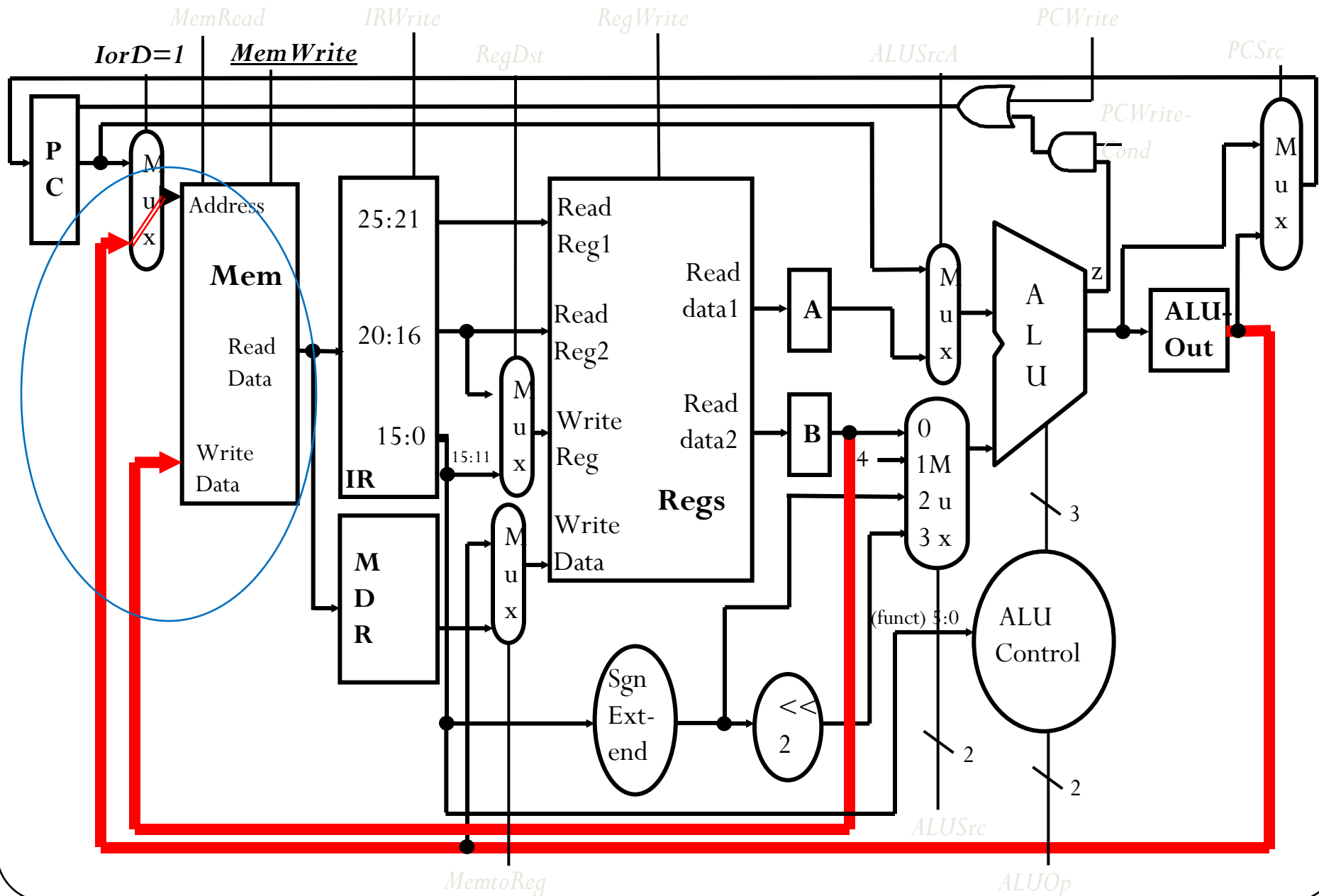


• How many times use ALU?

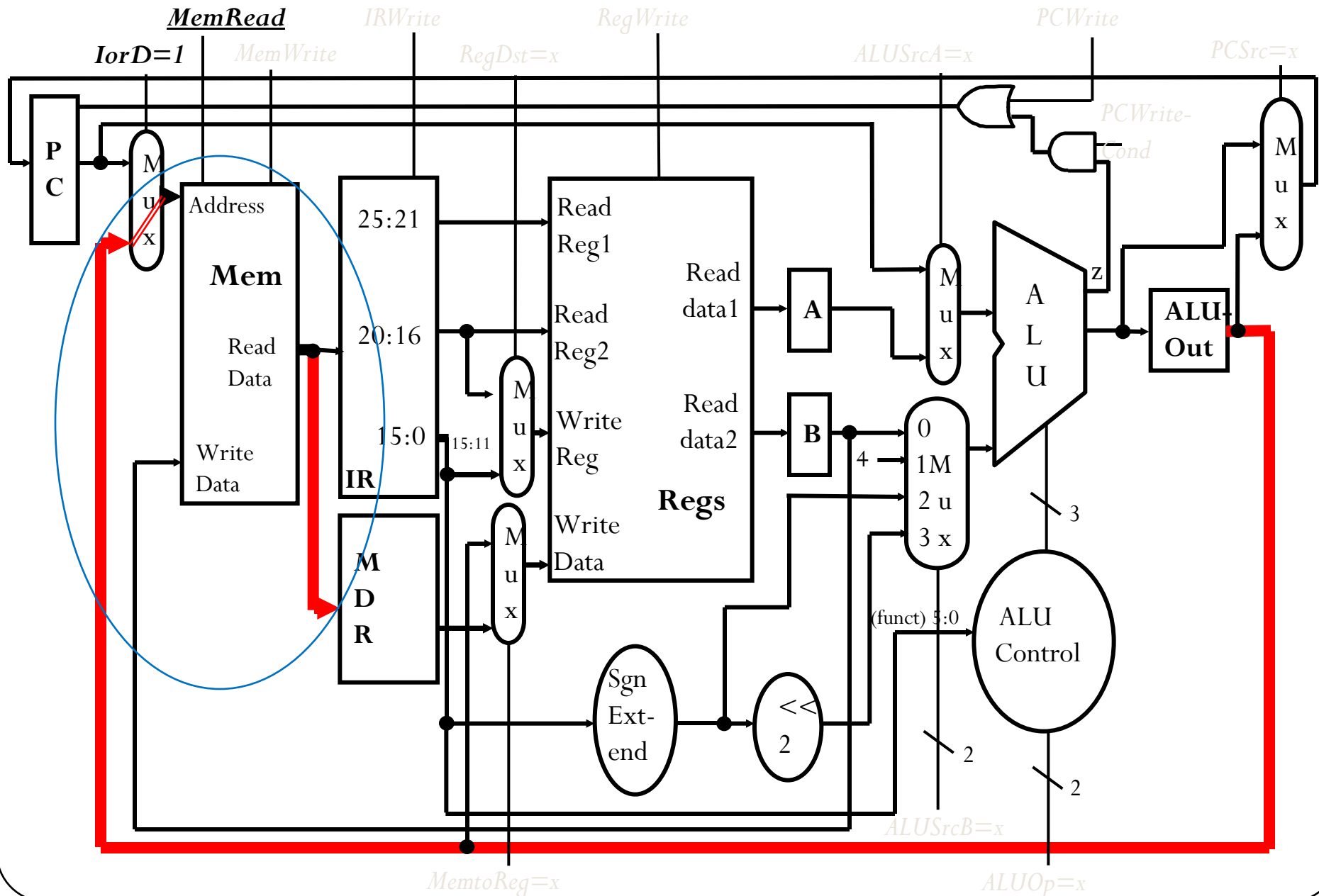
Cycle 3 Exec lw/sw: $ALUOut = A + sgn-ext(IR_{15:0})$



Cycle 4 Mem Access SW: $Mem[ALUOut] = B$



Cycle 4 Mem Acces $lw: MDR = Mem[ALUout]$



Cycle 5 Store lw: $Reg[IR_{15:11}] = MDR$

